

Value Objects

Modelando el dominio con tipos seguros

Carlos Terrero Orpí

basic_crud — Product domain

¿Qué es un Value Object?

Un **objeto inmutable** definido por **el valor de sus atributos**, no por una identidad.

- No tiene `id`
- Dos VOs con el mismo valor **son el mismo VO**
- Auto-validado en construcción
- Una vez creado, **no cambia**

"Si tengo dos billetes y los dos son de 5€, ¿son el mismo billete? Da igual: para pagar el café, son intercambiables."

El problema: primitive obsession

```
class Product
{
    private ?string $name = null;

    public function setName(string $name): void
    {
        $this->name = $name;    // ¿vacío? ¿espacios? ¿2 caracteres?
    }
}
```

- Cualquier `string` cuela: `""`, `" "`, `"a"`, `"\n"`
- La validación se duplica en cada `setName`, controller, command...
- El tipo `string` no comunica intención

La solución: **ProductName**

```
namespace App\Product\Entity\ValueObjects;

final readonly class ProductName
{
    private const int MIN_LENGTH = 3;

    public string $value;

    public function __construct(string $value)
    {
        $this->value = trim($value);
        if ($this->value === '') {
            throw ProductNameException::empty();
        }
        if (strlen($this->value) < self::MIN_LENGTH) {
            throw ProductNameException::tooShort($this->value, self::MIN_LENGTH);
        }
    }
}
```

Anatomía: **final readonly**

```
final readonly class ProductName
```

Modificador	Garantía
final	Nadie puede heredar y romper invariantes
readonly	Propiedades inmutables tras construir
public string \$value	Acceso directo sin getter ceremonial

PHP 8.2+ → el compilador es tu aliado. Sin trucos defensivos.

Validación en el constructor

```
public function __construct(string $value)
{
    $this->value = trim($value);

    if ($this->value === '') {
        throw ProductNameException::empty();
    }
    if (strlen($this->value) < self::MIN_LENGTH) {
        throw ProductNameException::tooShort($this->value, self::MIN_LENGTH);
    }
}
```

Always valid principle: si el constructor termina sin excepción, el objeto está en estado válido. Siempre.

Excepciones de dominio

```
namespace App\Product\Entity\ValueObjects\Exception;

final class ProductNameException extends InvalidArgumentException
{
    public static function empty(): self
    {
        return new self('Product name cannot be empty');
    }

    public static function tooShort(string $value, int $minLength): self
    {
        return new self(
            sprintf('Product name "%s" needs to be at least %d characters or more', $value, $minLength)
        );
    }
}
```

Named constructors → un solo tipo, dos formas de fallar bien descritas.

¿Por qué excepciones propias?

Genérica (<code>InvalidArgumentException</code>)	De dominio (<code>ProductNameException</code>)
Cualquier librería la lanza	Solo la lanza nuestro VO
Catch ambiguo	<code>catch (ProductNameException \$e)</code>
Mensaje en string suelto	Centralizado en la clase
Difícil de tracear	Habla el lenguaje del dominio

```
try {  
    new ProductName($input);  
} catch (ProductNameException $e) {  
    // sé exactamente qué falló y dónde  
}
```


Igualdad por valor

```
public function test_check_valueObject_has_the_same_value(): void
{
    $productNameFirst = new ProductName('Test Product');
    $productNameSecond = new ProductName('Test Product');

    $this->assertEquals($productNameFirst, $productNameSecond);
}
```

- `assertEquals` compara **valor** (no referencia)
- Dos `ProductName('Test Product')` son indistinguibles
- Es la esencia del Value Object

Normalización transparente

```
public function test_that_valueObjects_trim_the_name()  
{  
    $name = '    Test Product    ';  
    $productName = new ProductName($name);  
  
    $this->assertEquals('Test Product', $productName->value);  
}
```

El VO **limpia el input** antes de validar:

- " Test Product " → "Test Product"
- " " → vacío → excepción

Quien lo construye no tiene que recordar hacer `trim`.

Tests de validación

```
public function test_throws_ProductNameException_on_empty_name(): void
{
    $this->expectException(ProductNameException::class);
    $this->expectExceptionMessage('Product name cannot be empty');
    new ProductName('');
}

public function test_rejects_whitespace_only_name(): void
{
    $this->expectException(ProductNameException::class);
    $this->expectExceptionMessage('Product name cannot be empty');
    new ProductName(' ');
}

public function test_throws_ProductNameException_with_less_than_three_chars(): void
{
    $this->expectException(ProductNameException::class);
    $this->expectExceptionMessage('needs to be at least 3 characters');
    new ProductName('as');
}
```

Entity vs Value Object

	Entity (Product)	Value Object (ProductName)
Identidad	id (ULID)	Por valor
Mutabilidad	Sí (setName , setPrice)	Inmutable
Igualdad	Por id	Por valor
Ciclo de vida	Tiene historia	No tiene
Ejemplo	"Este producto concreto"	"El nombre 'Camiseta'"

Beneficios

1. **Type safety**: el compilador/analizador estático rechaza usos inválidos
2. **Validación en un solo sitio**: el constructor
3. **Intención explícita**: `ProductName` > `string`
4. **Tests focalizados**: pruebas el VO, no cada lugar que lo usa
5. **Refactor seguro**: cambios al concepto = cambios al VO
6. **Documentación viva**: el código describe el dominio

Pero no son la bala de plata

Todo lo anterior tiene un precio.

Un VO **resuelve un problema concreto**, la validación dispersa de un valor con invariantes, **no todos los problemas**.

Envolver por envolver es *over-engineering* con otro nombre.

Coste 1: boilerplate y proliferación

Un solo concepto = **tres ficheros** (VO + excepción + tests):

```
ProductName.php  
ProductNameException.php  
ProductNameTest.php
```

- Multiplícalo por cada campo: `Price`, `Sku`, `Slug`, `Stock` ...
- Más clases = más superficie que mantener y navegar
- El `->value` se filtra por todo el código que consume el VO

Para un valor **sin invariantes reales**, un `string` honesto es mejor que un VO ceremonial.

Coste 2: fricción en las fronteras

El mundo exterior habla en **primitivos**: JSON, formularios, columnas de BD.

```
// Entrada: hay que construir el VO desde el primitivo
$name = new ProductName($request->get('name'));

// Persistencia (MongoDB / Doctrine ODM):
// necesitas un type custom o mapear a mano
$document['name'] = $product->name->value;

// Salida (API/serialización): de vuelta a string
['name' => $product->name->value]
```

Cada borde del sistema es un punto de **conversión** ↔ **hidratación** que antes no existía.

Coste 3: invariantes que cruzan campos

Un VO solo conoce **su propio valor**. No puede validar reglas entre campos:

```
// ✗ ProductName no sabe nada del precio  
// "el precio con descuento debe ser < precio base"
```

- Esas reglas viven en la **entidad** o en un **domain service**
- El VO no te exime de tener lógica de dominio en otro sitio
- Riesgo: creer que "ya está todo validado" porque cada campo es un VO

Cuándo **no** usar un Value Object

Situación	Mejor opción
Valor sin invariantes (nota libre, flag)	Primitivo
Dato que solo pasa de largo (DTO transporte)	array / readonly DTO
Prototipo / CRUD trivial	Empieza simple, refactoriza luego
Regla que cruza varios campos	Entidad o domain service
El equipo no comparte el lenguaje DDD	Mide el coste de mantenimiento

Regla práctica

Crea un VO cuando hay **invariantes que proteger** y el concepto **se repite o tiene reglas propias**.

- ¿Cualquier valor del tipo es válido? → primitivo
- ¿Hay reglas, normalización o un concepto del dominio? → VO
- En la duda: empieza con el primitivo y **promociona a VO cuando duela**

El VO es una herramienta, no un dogma.

Próximos pasos en el proyecto

- [] `ProductPrice` + `ProductPriceException` — invariantes: > 0 , máximo 2 decimales
- [] Integrar VOs en `Product` (en vez de `?string`, `?float`)

Resumen

Un Value Object es:

inmutable · auto-validado · sin identidad · igual por valor

¿Por qué?

Modelar el dominio con tipos que **no pueden estar mal**.

¿Preguntas?

VO: `src/Product/Entity/ValueObjects/ProductName.php`

Excepción:

`src/Product/Entity/ValueObjects/Exception/ProductNameException.php`

Tests: `tests/Product/ValueObjects/ProductNameTest.php`